Лабораторная работа 3
Docker Compose

Часть 3: разделы 6-11 и дополнительное задание Adminer.

Студент: Есімханова Аружан
Группа: 25-ИСТК-2о
Дата: 22 октября 2026 года

Цель и выполненные проверки

Описать сервис и PostgreSQL одним Compose-файлом, проверить жизненный цикл данных, различие live/ready при отказе БД, режим разработки и просмотр таблиц через Adminer.

Пункт	Фактический результат
Compose up --build --wait	API и БД healthy; Docker Compose 5.5.1
Жизнь данных, три числа	3 → 3 → 2; отдельный учебный том Compose
БД остановлена	live 200; ready 503; /api/tasks 500; API healthy
Повторный запуск БД	Запросы снова 200 без перезапуска API
Проверки API	24/24 обязательных, 28/28 полных тестов; 73 assertions Postman
Дополнительное задание	Adminer 6.1.2: вход на db, просмотр projects и tasks
Секреты	.env игнорируется Git; config опубликован с удаленным паролем

14 технических проверок успешны. Отчет включает полный compose.yaml, журналы, ответы и два настоящих снимка Adminer. Накопительный чек-лист лабораторной 3 по-прежнему требует скриншоты консоли Visual Studio и Test Explorer; эти UI-пункты не объявляются подтвержденными.

Версия работы на GitHub

1. Compose: сервис API

Листинг 1, часть 1 из 2. Файл находится в корне репозитория. Фиксированное имя devops-labs делает имена ресурсов независимыми от папки и отделяет учебный стек от других проектов.

```
name: devops-labs

services:
  api:
    image: ${API_IMAGE:-devops-labs-demo-api}:${API_TAG:-local}
    build:
      context: .
      args:
        APP_VERSION: ${API_TAG:-local}
    ports:
      - "127.0.0.1:${API_PORT:-8088}:8080"
    environment:
      Storage: Postgres
      ConnectionStrings__Db: "Host=db;Database=demo;Username=demo;Password=${DB_PASSWORD:?Set
DB_PASSWORD in .env}"
      Chaos__Enabled: "true"
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    memswap_limit: 256M
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 256M
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
```

Ожидание, ресурсы и журналы

API запускается после успешного healthcheck PostgreSQL. Ограничения действительно применены: NanoCpus=1 000 000 000, Memory=268 435 456 и MemorySwap=268 435 456 байт. Совпадение Memory и MemorySwap означает отсутствие дополнительного swap. Политика перезапуска — unless-stopped, журналы json-file ограничены 10m × 3.

Chaos__Enabled=true включен согласно методичке для будущих лабораторных сбоев. Порт API опубликован только на localhost. Пароль требуется через DB_PASSWORD; демонстрационный пароль в compose.yaml не записывается.

2. Compose: БД, Adminer, том и сеть

Листинг 1, часть 2 из 2. Adminer включается профилем tools; обычный запуск поднимает API и БД.

```
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_DB: demo
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD in .env}
    ports:
      - "127.0.0.1:${DB_PORT:-15432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql
      - ./db:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U demo -d demo"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

  adminer:
    image: adminer:latest
    profiles: [tools]
    ports:
      - "127.0.0.1:${ADMINER_PORT:-8083}:8080"
    environment:
      ADMINER_DEFAULT_SERVER: db
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

volumes:
  pgdata:

networks:
  default:
    ipam:
      config:
        - subnet: ${COMPOSE_SUBNET:-10.254.54.0/24}
```

Параметры этой среды

Вместо занятого 8080 используется 8088; PostgreSQL опубликован на 15432. Контейнеры общаются на внутреннем порту 5432 по имени db. Явная свободная подсеть нужна из-за исчерпания стандартного пула Docker. Именованный том предыдущей работы сохранен отдельно; эксперимент использует devops-labs_pgdata.

3. Жизнь данных: три числа

На новом томе seed.sql создал два проекта. Через API добавлен проект Compose. После этого выполнены обе последовательности из пункта 9:

```
POST /api/projects  {"name":"Compose"}
docker compose down
docker compose up -d --wait
docker compose down -v
docker compose up -d --wait
```

Момент измерения	Количество и объяснение
До down	3: два учебных проекта и созданный Compose
После down и up	3: контейнеры и сеть пересозданы, именованный том остался
После down -v и up	2: старый учебный том удален, новый инициализирован из seed.sql; проекта Compose больше нет

Почему после сброса не ноль?

На пустом PGDATA entrypoint PostgreSQL снова выполняет db/seed.sql. Скрипт создает проекты «Курс DevOps» и «Домашние дела» и четыре задачи. Поэтому после полного сброса возвращаются именно два учебных проекта.

Резервная копия проверена

Перед down -v создан pg_dump -Fc. Архив скопирован за пределы репозитория и восстановлен в отдельную demo_restore: все три проекта, включая Compose, найдены. Перед сбросом также проверено, что в основном учебном томе нет посторонних проектов и задач. Том материала 05 не удалялся.

```
pg_dump -U demo -d demo -Fc -f /tmp/lab06-before-reset.dump
pg_restore -U demo -d demo_restore --no-owner /tmp/lab06-before-reset.dump
```

Разбор ручного стенда, пункт 6

Для воспроизведения только развертывания материала 05 нужны 7 команд и 21 вхождение опций: том, БД, проверка готовности, сборка API, сеть, подключение БД к сети и запуск API. Повторения ожидания, учебные ошибки и тесты в этот счет не включены. Compose объединяет описание в один файл и запуск в одну команду.

4. Сеть, остановка БД и готовность

Имя db разрешено из API-контейнера командой getent hosts db. Затем выполнен docker compose stop db и реальные HTTP-запросы:

Маршрут	Ответ при остановленной БД
/health/live	200 · процесс жив
/health/ready	503 · БД недоступна
/api/tasks	500 · SQL-запрос не выполнен

Почему API все еще healthy?

Dockerfile проверяет /health/live: процесс приложения продолжает отвечать. Готовность /health/ready отдельно включает PostgreSQL и возвращает 503. Запрос задач действительно обращается в БД, поэтому возвращает 500. Состояние healthy не означает, что зависимость доступна.

Что произошло после docker compose start db?

Готовность и запрос задач восстановились до 200. Время запуска API и счетчик перезапусков не изменились. depends_on определяет порядок запуска при up, а не непрерывно следит за зависимостью. Npgsql после восстановления БД снова смог выполнять запросы.

Секреты и .gitignore

Выполнена точная команда из чек-листа: Select-String '^\.env$' .gitignore. Вывод с заменой локального пути:

```
.gitignore:7:.env
```

Пароль находится только в локальном .env. В Git добавлен пустой .env.example. Rendered config проверен через docker compose config; для публикации пароль и абсолютные пути удалены. Сам config без обезличивания публиковать не нужно: он содержит подставленные значения.

Отсутствие DB_PASSWORD останавливает Compose до запуска. Это явное требование локального пароля вместо запасного demo из примера методички.

5. Разработка, тесты и общий чек-лист

API Compose остановлен, БД оставлена в контейнере. Windows-процесс dotnet занял тот же порт 8088 и подключился к PostgreSQL через localhost:15432. Запросы проектов и задач дали 200. После тестов Windows-процесс завершен, весь стек Compose возвращен в healthy.

Проверка	Фактический результат
Обязательная выборка	24/24 на PostgreSQL: 7 юнит + 17 интеграционных
Полный набор	28/28 на PostgreSQL: 7 юнит + 21 интеграционный
HTTP-сценарии	26 успешных запросов, включая очистку трех тестовых проектов
Postman CLI	24 запроса, 73 проверки, 0 ошибок

Тесты использовали отдельную demo_tests. Для числа 24 исключены четыре дополнительных случая DueDate; полный запуск содержит все 28. Код API не менялся. Результаты Postman оставлены локально, без облачной публикации.

Пункты 1-5 из предыдущей части

Без именованного тома новый PostgreSQL-контейнер получает другое хранилище и не видит notes. При повторном подключении именованного тома данные остаются, а seed.sql не выполняется: PGDATA уже инициализирован. В материале 05 подтверждено сохранение данных после перезапуска Windows/API-контейнера.

Две фактические ошибки предыдущей части и причины:

```
Unhandled exception. System.InvalidOperationException: Не задана строка подключения
ConnectionStrings:Db
Unhandled exception. Npgsql.NpgsqlException (0x80004005): Failed to connect to 127.0.0.1:5432
```

Первая: в Production не загружена строка из Development-конфигурации. Вторая: localhost внутри API-контейнера указывает на него самого, а не на контейнер PostgreSQL.

Открытые пункты

Скриншоты консоли Visual Studio и 24 зеленых случаев Test Explorer из накопительного чек-листа пока отсутствуют. Windows-запуск проверен через dotnet; ручное F5 не подтверждено. Новые пункты материала 06 подтверждены выводом команд и ответами, как допускают условия сдачи.

6. Дополнительно: Adminer

Adminer 6.1.2 запущен профилем tools на localhost:8083. Вход: PostgreSQL, сервер db, пользователь demo, база demo; пароль взят из локального .env. Просмотрены обе таблицы.

Рисунок 1. Таблица projects: учебные проекты и три проекта, оставленные коллекцией Postman после эксперимента 3 → 3 → 2.

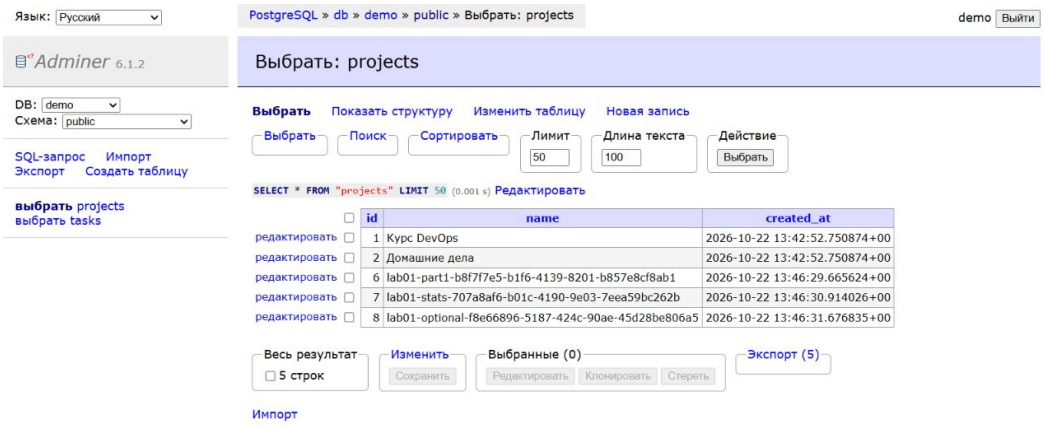
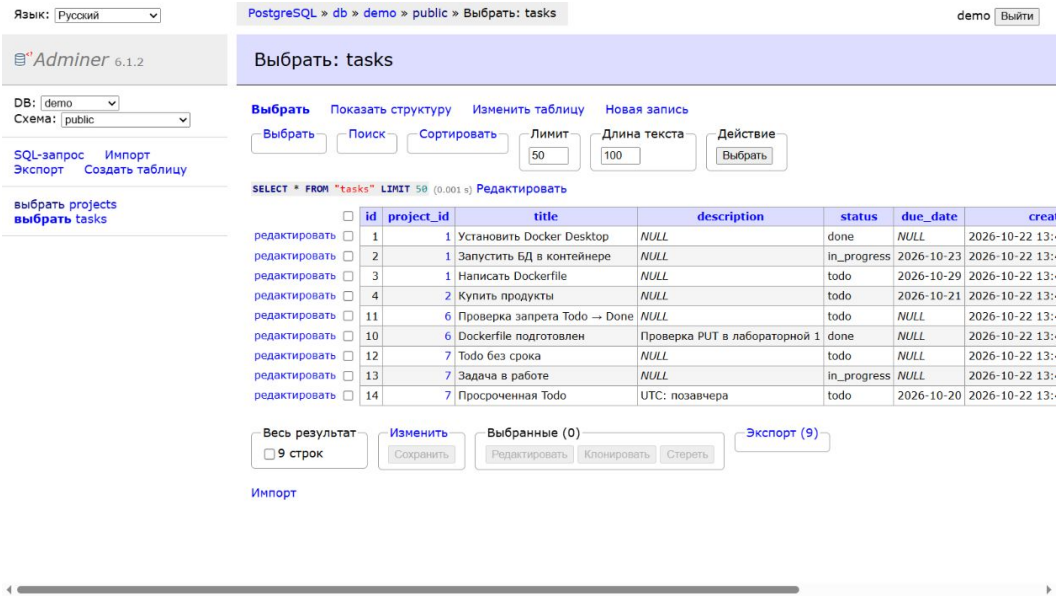


Рисунок 2. Таблица tasks: учебные задачи и данные коллекции Postman. Это настоящие снимки веб-интерфейса.



Какой хост указывать и почему не localhost?

db — DNS-имя PostgreSQL в общей сети Compose. localhost из Adminer означал бы сам контейнер Adminer. После демонстрации вход завершен, стек остановлен, именованный том сохранен.

Команды запуска и ответы · Все журналы и доказательства

Источник: материал 06, «Неделя 6 - Лаба 3. Данные и Docker Compose», разделы 6-11 и приложение из архива Devops (1).zip. Порядок запуска Compose · Официальный образ Adminer